

1. Introduccion

El Software Design Document (SDD) documenta la arquitectura, componentes, interfaces y datos del sistema antes de implementar. Su proposito es garantizar vision compartida, reducir ambigüedad y habilitar trazabilidad entre requisitos y solucion tecnica.

Referencias fundacionales: 'Software Architecture in Practice' (Bass, Clements, Kazman) y 'Documenting Software Architectures' (Clements et al.). Estas obras establecen el modelo de vistas multiples (modulos, componente-conector, asignacion) que permite a cada stakeholder obtener la informacion relevante sin saturarse.

1.1 Vistas Arquitectonicas

- Vista de Modulos: descomposicion en paquetes y modulos con responsabilidades claras.
- Vista de Componentes y Conectores: interacciones en tiempo de ejecucion (HTTP, llamadas a funciones).
- Vista de Asignacion: despliegue, mapeo a infraestructura, restricciones fisicas.

1.2 Arquitectura del Proyecto

El asistente inteligente sigue una arquitectura de capas simple:

```
final_project_AI_CUSTOM/  
  backend/                # Capa de servidor  
    server.py             # HTTP server (API REST)  
    assistant.py          # Orquestador pregunta->respuesta  
    knowledge.py          # Recuperacion RAG (scoring por terminos)  
    context_store.py      # Almacen de contexto CAG (pendiente)  
    cag.py                # Integracion contexto en respuesta (pendiente)  
  frontend/              # Interfaz web estatica  
    index.html            #  
    app.js                # Cliente HTTP contra backend  
    styles.css            #  
  data/                  #  
    knowledge_base.json   # Base de conocimiento (4 documentos)  
  tests/                 #  
    base/                 # Pruebas base (deben pasar siempre)  
    validation/           # Pruebas de validacion (CAG contract)
```

1.3 Decisiones Arquitectonicas Clave

ADR-01: Sin dependencias externas

El servidor usa la biblioteca estandar de Python (http.server, json, urllib) para eliminar la necesidad de pip install o entornos virtuales. Decision driven by simplicidad y portabilidad.

ADR-02: API REST sobre HTTP

Endpoints: GET /health, POST /api/ask, GET /api/context, POST /api/context. Formato JSON con CORS habilitado para el frontend.

ADR-03: Separacion RAG / CAG

knowledge.py maneja recuperacion de conocimiento general (RAG). context_store.py + cag.py manejan contexto persistente del usuario (CAG). Separacion limpia de responsabilidades.

2. Atributos de Calidad

- Mantenibilidad: modulos con una unica responsabilidad (server, assistant, knowledge, context_store).
- Testeabilidad: servidor con puerto dinamico (port=0) para tests aislados.
- Extensibilidad: nueva logica CAG se agrega sin tocar server.py mas que conectar los handlers.
- Portabilidad: Python stdlib solo -> corre en cualquier SO con Python 3.8+.

3. Flujo de Datos: /api/ask

```
Frontend --POST /api/ask {user_id, question}--> server.py
-> ExamRequestHandler.do_POST()
-> assistant.answer_question(user_id, question)
  -> knowledge.retrieve_snippets(question)      # RAG
  -> load_knowledge_base()                      # Lee JSON
  -> scoring por terminos, top-2
  -> assistant arma respuesta con sources
-> devuelve JSON {answer, sources, context_used}
<- JSON response --> Frontend renderiza
```

4. Referencias

[Software Architecture in Practice \(Bass, Clements, Kazman\) - Addison-Wesley](#)

[Documenting Software Architectures \(Clements et al.\) - Addison-Wesley](#)

[SEI - Carnegie Mellon: Software Architecture Resources](#)

[ISO/IEC 42010: Systems and software engineering - Architecture description](#)

[ACM Digital Library - 'software architecture' 'architectural decisions'](#)

Nota: El SDD debe actualizarse cuando se implemente CAG completamente, agregando las nuevas vistas y decisiones asociadas al almacen de contexto.